
Journal of Graph Algorithms and Applications
http://jgaa.info/ vol. 0, no. 0, pp. 0–0 (0)
DOI:

Sharing Work Within Update Batches for Exact Dynamic Counting of Triangles, 4-Cycles and 5-Cycles: An Experimental Study

Hamzah Asyrani Sulaiman[1] *Shafina binti Abd Karim Ishigaki*[1]
Asniyani Nur Haidar binti Abdullah[2] *Nazreen Abdullasim*[1]
Mohamad Lutfi Dolhalit[1] *Abdullah Bade*[3]

[1]Department of Interactive Media, Faculty of Information and Communication Technology, Universiti Teknikal Malaysia Melaka, Hang Tuah Jaya, 76100 Durian Tunggal, Melaka, Malaysia
[2]Department of Software Engineering, Faculty of Information and Communication Technology, Universiti Teknikal Malaysia Melaka, Hang Tuah Jaya, 76100 Durian Tunggal, Melaka, Malaysia
[3]School of Science and Technology, Universiti Malaysia Sabah, Jalan UMS, 88400 Kota Kinabalu, Sabah, Malaysia

Submitted:	Accepted:	Published:
Article type: Regular Paper		Communicated by:

Abstract. We study exact sequential counting of triangles, 4-cycles and 5-cycles in a graph that receives insertions and deletions in batches. We ask whether sharing work between the updates of one batch beats the strongest kernels that do not share. The sharing kernel groups a batch into stars and builds one row of path counts at the centre of each star, which every update of the star reuses. It is compared with a per-update scan, wedge tables, a heavy-light mixed table, an affected-subgraph recount and, for 4-cycles, a published batch algorithm. Every count is checked to be exact, and every cost of upkeep is included in the measured time. A cell is one stream, batch size and pattern, and it passes when the slowest repeat of the sharing kernel takes at most 0.67 times the fastest repeat of the best other kernel. On 16 real-timestamp streams, six with recorded deletions, sharing passes on 28 of 50 cells for 5-cycles, on none of 48 for triangles, and on none of 12 for 4-cycles, which were timed on three streams. It fails on the one stream held out until the end, and with a scan 10% faster only 16 of the 50 cells would pass. On most cells, the kernel with the fewest operations is not the fastest. A linear model that prices each kind of operation, fitted on three small streams, names the fastest kernel on 63 of 64 cells of twelve unseen streams, against 21 for a single price per operation. All timing is from one machine.

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

E-mail addresses: asyrani@utem.edu.my (Hamzah Asyrani Sulaiman) shafina@utem.edu.my (Shafina binti Abd Karim Ishigaki) asniyani@utem.edu.my (Asniyani Nur Haidar binti Abdullah) nazreen.abdullasim@utem.edu.my (Nazreen Abdullasim) lutfi@utem.edu.my (Mohamad Lutfi Dolhalit) abb@ums.edu.my (Abdullah Bade)


This work is licensed under the terms of the CC-BY license.

1 Introduction

Counting triangles and other short cycles in a graph that changes over time is a basic task in the analysis of temporal networks. A triangle count gives the clustering of a social or communication network. Counts of 4-cycles, called butterflies in bipartite graphs [38], and of 5-cycles describe structure that triangles cannot. When the graph changes one edge at a time, the count can be kept exact by adding, for each update, the number of cycles that the updated edge closes. Hanauer et al. [16] engineered such single-update counters on 56 instances and found that, for triangles, a plain scan of the lower-degree endpoint with no auxiliary structure is the fastest choice.

Real streams do not arrive one edge at a time. They arrive in batches, and the updates of a batch often share an endpoint: a user who answers several questions within one batch adds several edges at the same vertex. The work done for one of these updates can then be reused for the next. DPTL+ [40] reuses one neighbourhood bitmap per vertex across a batch for listing triangles, BFCB-IG [38] reuses one row per vertex for counting butterflies, and Cheetah [42] groups the updates of a batch into stars for general patterns and reports gains that grow with the size of the pattern.

Those gains were measured against systems that do not share, and not against the strongest sequential exact counter for each pattern. Cheetah's authors note that a pattern-aware system without sharing stays close to Cheetah on triangles, 4-cycles and 5-cycles on some of their inputs [42]. So it is still open whether sharing work inside a batch pays against a well-implemented scan or wedge table, once every cost of grouping and upkeep is counted.

We ask that question for triangles, 4-cycles and 5-cycles, under batches of insertions and deletions, on real timestamped streams. The sharing kernel builds one row of path counts at the centre of each star and adjusts it for each update of the star. It is compared with a per-update scan, two wedge tables, a heavy-light mixed table, a recount of the affected subgraph, and at four vertices a port of a published batch algorithm. Every count is checked to be exact after every batch. The test for a win was fixed before any timing run: on a cell, the slowest repeat of the sharing kernel must take at most 0.67 times the fastest repeat of the best other kernel.

The answer depends on the size of the pattern. For 5-cycles, the sharing kernel passes this test on 28 of 50 scored cells, over 14 of 16 streams, including streams with recorded deletions. For triangles it passes on none of 48 cells, and for 4-cycles on none of 12 cells on three streams, although for triangles it is faster than the scan by a median of 13% on the twelve newest streams. The one stream held out until the end of the study gives no pass, and neither does the German Wikipedia stream. At five vertices, the best kernel other than sharing is the scan on 43 of the 50 cells, and a wedge table is the best other kernel on none.

Counts of operations predict none of this. On 60 of the 76 cells of the thirteen newest streams, the kernel with the fewest operations is not the fastest. At five vertices a wedge table or the recount usually does the fewest operations while the sharing kernel is fastest, because a read or write of a hash table costs several times a step along an adjacency list. A linear model that prices each kind of operation separately, fitted on three small streams and read on twelve larger streams it never saw, names the fastest kernel on 63 of 64 cells, where a single price per operation names 21. In both fits that we report, a read or write of a wedge table costs 1 to 5 ns and a step along a row or list 0.05 to 0.6 ns.

This paper makes three contributions.

1. An engineered and measured star-row kernel for exact batch-dynamic counting of 5-cycles, with all costs of reducing the batch, grouping it and maintaining every structure included in the measured time.

2. A rule for when sharing pays, tested on 16 real-timestamp streams: at five vertices on 28 of 50 cells, more often at batches of 1,000 events than of 100, and at three or four vertices on none, with every cell listed in Appendix A.

3. A cost model priced by kind of operation that explains why operation counts choose the wrong kernel, together with its limits.

We do not claim the idea of reusing a row across the updates of a batch, which DPTL+, BFCB-IG and Cheetah publish, nor the argument that sequential updates count each cycle exactly once, nor heavy-light partitioning or wedge indexes, which are prior work.

Section 2 places the work beside the closest prior work. Sections 3 to 5 define the problem, the kernels and the measurement. Section 6 gives the results, Section 7 the cost model, Section 8 the sensitivity analysis, and Section 9 the limits and the conclusion.

2 Related Work

Exact Counting Under Single Updates. Eppstein and Spiro [13] maintain the counts of all 3-vertex subgraphs in amortized time proportional to the h-index of the graph, and Eppstein et al. [12] extend this to 4-vertex subgraphs. Kara et al. [19] maintain a triangle count in $O(\sqrt{m})$ amortized time per update, which is optimal under the OMv conjecture, and Lu and Tao [26] give bounds that depend on the arboricity. Hanauer, Henzinger and Hua [17] count all connected 4-vertex subgraphs with tables that hold only the paths through low-degree vertices, and count through high-degree vertices from the adjacency lists; our mixed tables follow this rule. Hanauer et al. [16] engineer these approaches on 56 instances and find that, for triangles, scanning the lower-degree endpoint is the fastest choice, while heavy-light tables pay for 4-cycles. IncGraph [5] computes the change in all graphlet counts up to five vertices for each edge by local enumeration. None of these works considers batches, and only IncGraph reaches five vertices. Hanauer, Henzinger and Schulz [18] survey the engineering of fully dynamic graph algorithms more broadly.

Batches. Dhulipala et al. [10] count triangles and k-cliques under batches in parallel, with heavy-light tables, and make each triangle count once by dividing by its number of updated edges. Liu et al. [25] maintain a low out-degree orientation under batches and bound the work of k-clique counting through it. Makkar et al. [27] count triangles on the GPU under batches, by inclusion and exclusion over per-edge intersections, with no work shared between the edges of a batch, and EDTC [39] is a recent GPU counter for dynamic triangles. Ediger et al. [11] group a batch by source vertex, but for the update of the data structure rather than for the count. ParaDyMS [20] recounts motifs in the modified part of the network after each batch, which is the idea of our recount kernels A4 and A5.

Sharing Inside a Batch. Three published methods reuse one row per vertex across the updates of a batch. DPTL+ [40] lists the triangles that a batch adds or removes, in parallel, and builds one bitmap of the neighbourhood of each pivot vertex, which every batch edge at that vertex reuses. BFCB-IG [38] counts butterflies, the 4-cycles of a bipartite graph, by building one row of 2-path counts per start vertex over the affected subgraph. Cheetah [42] groups each batch into stars for general patterns, caches neighbour lists per star, and reports a mean gain of 1.12 times for triangles and 2.5 to 2.75 times for larger patterns on a cluster of 16 nodes, against Tesseract [3], with graph-update time left out. Gopher [41] shares the set operations within one update instead,

Work	What it does	What this paper adds
Hanauer et al. [16]	Exact counters for single updates, up to 4 vertices, on 56 instances	Batches, sharing within a batch, and 5-cycles
Dhulipala et al. [10]	Parallel batch counting of cliques with heavy-light tables	Sequential counting of cycles with star rows, against tables, with all costs counted
DPTL+ [40]	One bitmap per pivot reused across a batch, for parallel triangle listing	Counting without listing, a degree guard, and up to 5-cycles
BFCB-IG [38]	One row of 2-path counts per start vertex, for butterflies in bipartite graphs	Odd cycles and general graphs, and a row adjusted for each spoke
Cheetah [42]	Star grouping with reuse for general patterns, on a cluster, update time left out	A kernel that counts by formula without listing, sequential, against the strongest baselines, with every cost inside

Table 1: The closest prior work.

and its authors report that this beats Cheetah's sharing across updates. The idea that this paper measures is therefore published, for triangles, butterflies and general patterns.

Continuous Subgraph Matching. General systems for continuous subgraph matching are compared by Sun et al. [37], who find that no algorithm wins everywhere and that index updates can dominate the time. RapidFlow [36] reduces each update to batch matching over the affected region, GAMMA [33] matches under batches on the GPU and removes duplicates by a total order on the updates, and Mhedhbi et al. [28] share operators between delta queries. Ammar et al. [2] state the delta-query rule: each delta query reads the new versions of the earlier relations and the old versions of the later ones, and together these queries give exactly the change in the output. These systems list matches of general patterns. Hanauer et al. report that dedicated counters are faster on triangles [16]. Table 1 sets out the closest work.

Static Counting. Chiba and Nishizeki [9] list triangles and 4-cycles in time proportional to the arboricity times the number of edges, Alon, Yuster and Zwick [1] count cycles of a given length, ESCAPE [31] counts all 5-vertex subgraphs with a per-vertex row, and Shi, Huang and Shun [35] count 5-cycles in $O(m\alpha^2)$ work. Our recount kernels use the first and the last of these.

Timestamped Streams and This Journal. Chanchary and Maheshwari [7] answer time-windowed queries on timestamped event graphs, including counts of 2-paths and quadrangles. Engineering studies of published algorithms, with experiments on real instances, appear in this journal for hypergraph matching [15], 2-packing sets [4] and synchronized planarity [14], and this paper follows their form.

3 Problem and Model

Graphs and Streams. A stream is a sequence of events (t, u, v, σ) with a timestamp t, two vertices and an operation σ that inserts or deletes the undirected edge $\{u, v\}$. Self-loops are

dropped, direction is dropped, and the graph is kept under set semantics: inserting a present edge or deleting an absent one changes nothing. This is the cleaning of Hanauer et al. [16], whose instance set our streams come from (Section 5). Every stream starts from the empty graph.

Three Labels. Each stream carries one of three labels, and every table states it. *Real timestamps, insert-only*: the events of a temporal file in time order, with no deletion. *Real timestamps, synthetic expiry*: the same insertions, each followed by the deletion of its edge fourteen days later, with deletions before insertions at equal times, as in Hanauer et al. [16] after Chen et al. [8]. *Real timestamps, real deletions*: the link streams of six Wikipedia editions from KONECT [22, 32], whose deletions are recorded events, in which 16% to 28% of the events are deletions. No fully synthetic stream enters any result.

Batches. A stream is cut into consecutive batches of c events, with $c = 100$ and $c = 1{,}000$. A batch is first reduced to its net effect: an edge whose state at the end of the batch equals its state at the start is dropped, and every other edge becomes one net insertion or one net deletion. After each batch an algorithm reports the exact number of copies of C_p, the cycle on p vertices, in the current graph, for $p \in \{3, 4, 5\}$. An answer that differs from the true count on any batch is a failure, whatever its time.

The Delta of One Update. All algorithms in this paper apply the net updates of a batch one at a time and add up their deltas. For an update of the pair $\{u, v\}$, the delta is $\pm$ the number of p-cycles that the pair closes, counted in the graph without the pair, that is, before the insertion or after the deletion. Writing $N(x)$ for the neighbourhood of x in that graph, $d(x) = |N(x)|$ for its degree, and $W[a, b] = |N(a) \cap N(b)|$ for the number of common neighbours,

$$\Delta_3(u, v) = W[u, v],$$

$$\Delta_4(u, v) = \sum_{a \in N(u)} W[a, v],$$

$$\Delta_5(u, v) = \sum_{b \notin \{u, v\}} A[b]\, C[b] \; - \sum_{t \in N(u) \cap N(v)} \big(d(t) - 2\big),$$

where $A[b]$ counts the paths (u, a, b) with $a \neq v$, and $C[b]$ the paths (v, c, b) with $c \neq u$. The sum $\sum_b A[b]C[b]$ counts every triple (a, b, c), and the only triples that do not form a path (u, a, b, c, v) on five distinct vertices are those with $a = c$. Such a vertex $t = a = c$ lies in $N(u) \cap N(v)$, and each of its neighbours other than u and v gives one such triple, which is the second term. Applying the updates in sequence counts every cycle created or destroyed by the batch exactly once, because the deltas telescope. This is the delta-query rule of Ammar et al. [2] with one update per step, and we do not claim it.

All Costs Count. The time of a batch includes reducing the batch to its net updates, grouping it where an algorithm groups, updating the graph, and maintaining every auxiliary structure: wedge tables, lists of heavy vertices, and the switch of a vertex between heavy and light. No structure is built ahead of the stream, and no algorithm reads beyond the current batch.

Role	$p = 3$	$p = 4$	$p = 5$
Per-update scan	S3	S4	R2
Sharing, star row	**P3**	**P4**	**PR**
Wedge table, tuned layout	T3	T4	WT
Wedge table, flat			WJ
Mixed table, hubs scanned	M3	M4	M5
Affected-subgraph recount		A4	A5
Published batch sharing, port		B4	

Table 2: The arms. Sharing arms in bold.

4 Kernels

Table 2 lists the kernels, which we also call arms. Each arm computes the deltas of Section 3 in its own way, and all arms of a pattern give the same count after every batch.

Scan. S3, S4 and R2 compute each delta from the adjacency lists alone. R2, at $p = 5$, builds the row A at the endpoint of lower degree and streams the other side through it, so that $\sum_b A[b]C[b]$ is read without storing C.

Sharing by Star Rows. A batch's net updates often share an endpoint: a user who posts several answers in one batch adds several edges at one vertex. The sharing arms group the net updates of a batch into stars. Each update goes to the endpoint at which the batch has more updates, its pivot, and the updates at one pivot x form a star whose spokes are the other endpoints. For a star at x, PR builds the row A at x once and adjusts it by each spoke in turn, so that each spoke sees the graph with the earlier spokes already applied. Each spoke then streams its own side through the shared row, as R2 does. For a star with one spoke PR runs R2 itself, so PR can lose to R2 only by the cost of grouping. Figure 1 shows one batch of a real stream: its largest star has 24 spokes, and the shared row costs 6,097 operations where one row per spoke costs 64,445.

P3 and P4 apply the same idea to triangles and 4-cycles. They carry a guard that reads only degrees, has no tuned parameter, and whose own operations are counted: a star keeps its row only if the row costs less than the scans it replaces, and otherwise its spokes run as S3 or S4. The guard is needed because S4 walks from the endpoint of lower degree, while a row is always built at the centre. In early tests, a star at a hub with a few small spokes made P4 slower than S4. PR has no guard. A guard could only make PR faster, so leaving it out can only understate the gain of sharing. Reusing one row per vertex across a batch is published for triangle listing by DPTL+ [40] and for butterfly counting by BFCB-IG [38], and grouping a batch into stars for general patterns by Cheetah [42], together with the finding that triangles gain least. We do not claim either.

Wedge Tables. T3, T4, WT and WJ keep $W[a, b]$ for every pair of vertices with a common neighbour, and update it on every edge edit. A delta then becomes a sum of table reads: $W[u, v]$ at $p = 3$, a sum over $N(u)$ at $p = 4$, and at $p = 5$

$$\Delta_5(u, v) = \sum_{\substack{a \in N(u),\ c \in N(v) \\ a \neq c}} W[a, c] \;-\; \tau\,(d(u) - 1) \;-\; \tau\,(d(v) - 1), \qquad \tau = |N(u) \cap N(v)|,$$

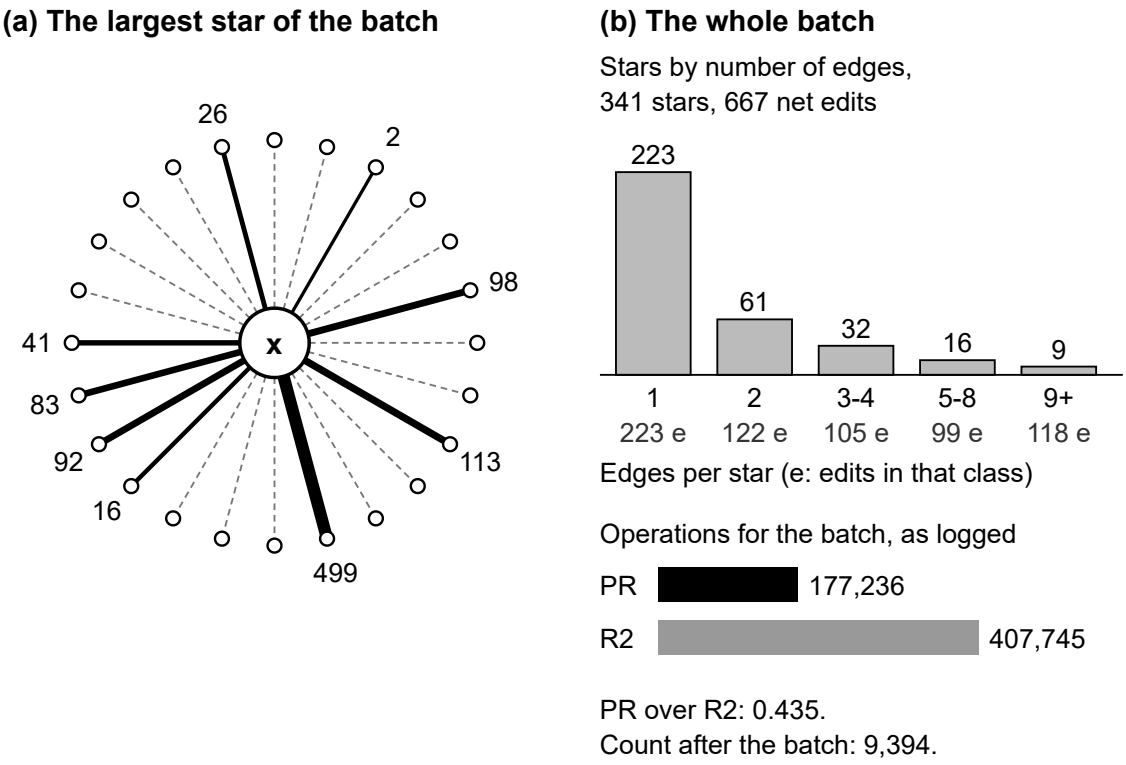


Figure 1: A worked batch: batch 465 of the 1,454 batches of sx-askubuntu with synthetic expiry, at 1,000 events per batch and $p = 5$. The 1,000 events net to 667 edits. Panel (a) shows the largest star of the batch, 24 insertions at one vertex, each edge labelled with the 5-cycles it adds: PR builds one row at the centre for 6,097 operations, where R2 builds one row per edge for 64,445. Panel (b) shows the whole batch: its 341 stars by number of edges, and the operations of PR and R2, 177,236 against 407,745. The batch was chosen by a rule fixed before any batch was read: the median of PR over R2 among the batches where PR does fewer operations. Operation counts from the count build, not time: on the clock, PR over R2 on this cell is 0.588, a smaller gain than the 0.435 in operations.

with W taken in the graph without the pair. WJ stores the table in one flat hash map. WT holds the same entries in one open-addressing table per vertex, with each pair stored in both of its tables, so that a join reads only the tables of the shorter list. Its slots are four bytes, they are searched four at a time with SSE2, and the join switches from probes to one pass over the table against a bitset when that is cheaper. The layout of WT was chosen by trial on synthetic R-MAT streams [6], so no stream of this paper shaped it. T3 and T4 use WT's table.

Mixed Tables. M3, M4 and M5 follow the idea of Hanauer, Henzinger and Hua [17]: the table holds only the wedges through light centres, so a hub pays no upkeep, and the wedges through hubs are counted from the adjacency lists at query time. A vertex turns heavy above degree 2θ and light again below θ, so a switch costs at most $(2\theta)^2$ pairs and is paid for by at least θ updates. Writing H for the set of heavy vertices, the hub term at $p = 5$ is $\sum_{h \in H}(A[h]C[h] - T[h])$, with $A[h] = |N(u) \cap N(h)|$, $C[h] = |N(v) \cap N(h)|$ and $T[h] = |N(u) \cap N(v) \cap N(h)|$. One value of the threshold θ per pattern was chosen on the engineering streams of Section 5, before any evaluation run of a mixed table, from $\{16, 32, \ldots, 1024\}$: 16 at $p = 3$ and $p = 4$, and 128 at $p = 5$. Section 8 gives M5 its best θ per cell instead.

Recount and Published Sharing. A4 and A5 recount the cycles inside the ball of radius r around each changed edge, $r = 1$ at $p = 4$ and $r = 2$ at $p = 5$, in the union of the old and new graphs, and take the difference. The static counters are Chiba and Nishizeki's wedge pairing [9] at $p = 4$ and Algorithm 1 of Shi, Huang and Shun [35] at $p = 5$. B4 ports the batch 4-cycle algorithm BFCB-IG of Wang et al. [38] from bipartite to general graphs. It splits a mixed batch into deletions and then insertions, as BFCB-IG does, and builds one row per start vertex that every edit at that vertex reads. We know of no batch algorithm of this form for 5-cycles, so B4 has no counterpart at $p = 5$. There is no recount at $p = 3$: in an earlier operation count of ours on six of the streams of Section 5, a triangle update cost 5.9 to 13.9 operations, and a recount cannot compete with that.

Exactness. An oracle test compares every arm of every pattern with a brute-force count over vertex tuples, after every batch, on random small graphs whose batches hold deletions, hubs, whole stars, repeated pairs and no-op edits. Mixed arms run there at θ down to 1, so vertices switch on graphs this small. Fourteen mutants, each a plausible way to get an arm wrong, must each fail the test, and so must a naive per-edge sum, which counts a cycle once for each of its changed edges. On the streams, every arm's count matches every other arm's after every batch, in every timing repeat and in every count build.

5 Method of Measurement

Streams and Their Roles. Twenty streams enter the study, in four sets that were fixed before any of them was timed. The *engineering* streams, CollegeMsg [29], email-Eu-core-temporal and sx-mathoverflow [30], were used to choose θ and to fit the cost model of Section 7, and never enter a verdict. The *evaluation* streams, sx-askubuntu, sx-superuser and wiki-talk-temporal [30, 23], decide the first verdict. These six streams and sx-stackoverflow come from SNAP [24]. The *held-out* stream, sx-stackoverflow, was chosen before any timing run and timed once, at $p = 5$ only. Hanauer et al.'s set [16] has 56 instances, 18 of them real streams, and four of those 18 are in the first three sets. The *new* streams are 13 of the other 14, chosen by a rule written before their sizes were read: six Wikipedia link streams with real deletions from KONECT, two Reddit hyperlink

streams from SNAP [21], and five temporal files from Network Repository [34]. Each is cut to its first 2,000,000 events, which is Hanauer et al.'s own prefix. One of them, soc-youtube-growth, has a single timestamp in that prefix and is reported but never scored. The 38 Kronecker streams of Hanauer et al.'s set are left out as fully synthetic. Expiry and insert-only variants run on every stream without real deletions, at both batch sizes. We call one combination of a stream, its variant, a batch size and a pattern size a *cell*.

Operations First. Each arm exists in two builds from one source. The count build counts every unit operation: a list element scanned, a lookup of an edge in the graph's hash map, a row read or write, a table read or write, a degree read, a graph edit, and each step of reducing and grouping the batch. Operation counts are deterministic and do not depend on the machine, and they were read before any clock. The timing build has no counters, because a counter in a hot loop is a store the compiler cannot move, and on a synthetic stream the counters cost R2 and PR about 15% and WJ about 1%. All seconds in this paper come from the timing build, and all operation counts from the count build. Figure 2 shows the count and the operations of five kernels over a whole stream.

The Clock. All runs are on one machine, an Intel Core Ultra 7 265 with 31 GB of memory and a 30 MB last-level cache, under Windows 11, with GCC 16.2 at `-O2`, linked statically. One process runs at a time, pinned to one performance core, logical CPU 8. On a cell whose fastest arm takes under 60 s each arm runs five times, and otherwise three, with the arms interleaved across repeats in a rotated order. An arm more than five times slower than the fastest on a cell runs once. A process stops at 1,200 s or 16 GB of committed memory, and A4 and A5 stop at 300 s. An arm stopped by time has not finished. It is never counted as a win, and in comparisons its time is its limit. An arm stopped by memory is out of that cell.

The Rule, Fixed Before the Clock. Let *share* be P3, P4 or PR, or at $p = 4$ the faster of P4 and B4. Let *best* be the fastest of the other arms, leaving out B4. An arm stopped by its time limit counts in this comparison with its time set to the limit. A cell *passes* when the slowest repeat of share is at most 0.67 times the fastest repeat of best. Comparing the slowest repeat with the fastest is deliberate. In an earlier experiment of ours, a cell passed when the fastest repeats were compared, by a difference of one clock tick, and failed when the slowest of three repeats was used. A cell's *score* is the ratio of the medians. A cell is *scored* if share and at least one other arm finished or were stopped by time. The verdict at $p = 5$ is Go if at least a third of the scored cells pass, on at least two streams, with at least two expiry cells among them. These thresholds, and the treatment of stopped arms, were fixed before the first timing run and were not changed after it.

What Exclusive Means. The machine is shared with other projects, and timing ran only while it was reserved for this study: no other long or parallel process ran during a timing run. The machine was not silent. Scheduled tasks of another project ran every quarter hour, a status mail ran every two hours, and desktop sessions stayed open. After each timing run, every timed process was checked against the recorded start times of these tasks, and repeats that overlapped one were compared with clean repeats of the same arm on the same cell. In the last timing run, for the cost model, one of 790 scored repeats overlapped a known task, and it ran 0.2% over its arm's median. Section 8 gives the same comparison for the runs on the new streams.

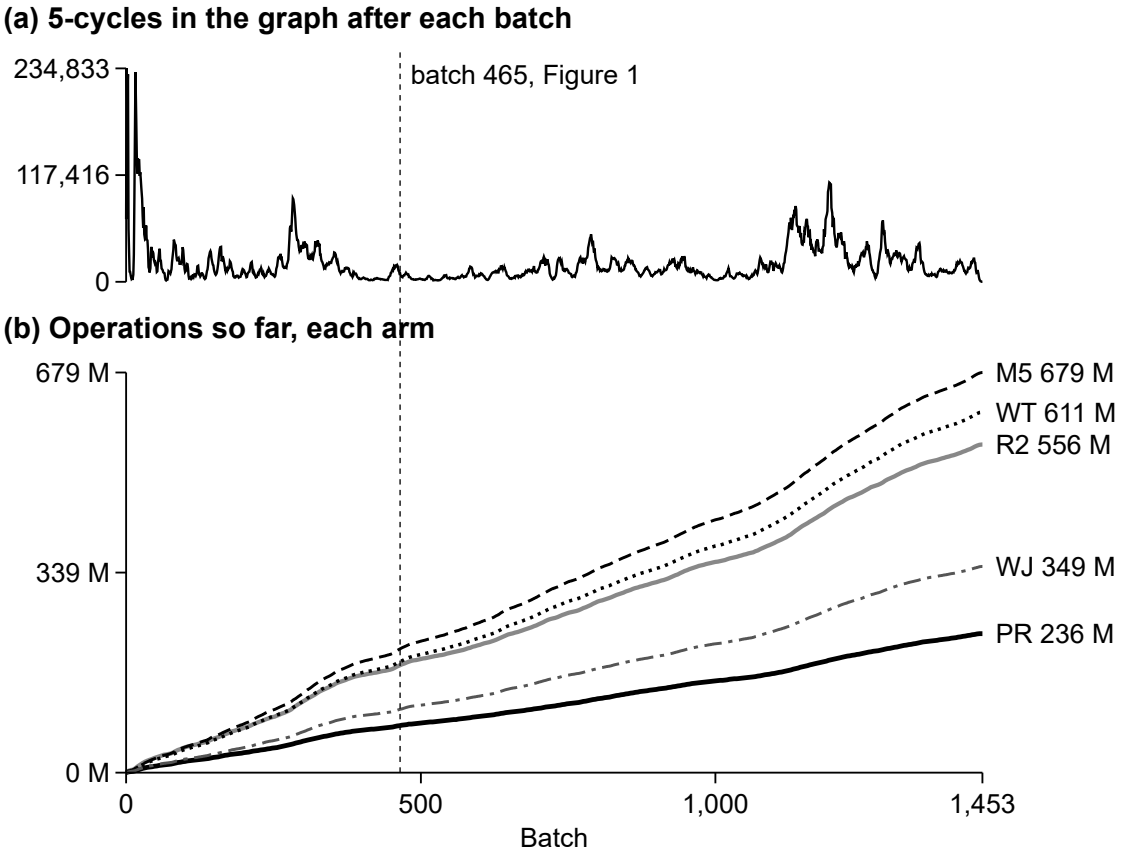


Figure 2: The same stream over all 1,454 batches: (a) the 5-cycle count after each batch, and (b) the cumulative operations of PR, R2, WT, WJ and M5. Every kernel's count agrees on every batch. The expiry rule empties the graph by the end of the stream. Operation counts from the count build, not time. On this cell PR also does the fewest operations; on the insert-only cells of the sx streams WJ does fewer operations than PR and is slower, so this cell is not the general case.

Cores. To test whether a verdict depends on the core, the 24 cells nearest the pass line were timed again on a performance core and on an efficiency core of the same processor, in one timing run. The two kinds of core differ in design and in their first two cache levels, and share the last-level cache and memory. So this is not a second machine, and the paper does not use it as one.

6 Results

First Verdict. On the three evaluation streams and the held-out stream, the verdict by the rule fixed before the first timing run is Go at $p = 5$: 6 of 14 scored cells pass, on all three evaluation streams, and 4 of the passes are expiry cells. Two cells of the held-out stream are not scored, because no kernel finished them. At $p = 3$ no cell of 12 passes, and the median score is 1.098, so sharing is slower than the best other kernel there. At $p = 4$ no cell of 12 passes either, with a median score of 1.299, and the best other kernel is the scan S4 or the mixed table M4. The guarded star-row kernel P4 takes 0.094 to 0.323 of the time of the BFCB-IG port on every cell.

Twelve New Streams. On twelve streams that no kernel, parameter or reading of this study had touched before, 22 of 36 scored cells pass at $p = 5$, on 11 of the 12 streams. Pooled with the first set, 28 of 50 cells pass, on 14 of 16 streams, so the verdict holds on the larger set. Table 3 gives the counts by label, and Appendix A lists every cell. The passes fall under all three labels: 9 of 12 insert-only cells, 6 of 12 expiry cells and 7 of 12 real-deletion cells pass on the new streams, so recorded deletions do not prevent a pass. Figure 3 shows every stream with its label, its size and its passes.

Batch Size. Passes depend on the batch size. At batches of 1,000 events, 19 of 25 pooled cells pass, and at batches of 100, 9 of 25 pass. A larger batch holds more updates at each centre, so the row built once at that centre is reused by more spokes. Figure 4 shows the score of every scored cell against the pattern size, for both batch sizes.

Where It Fails. Two streams give no pass: the held-out stream, sx-stackoverflow, and the German Wikipedia stream, dewiki. On the expiry cells of sx-stackoverflow the score is 0.846 and 0.978, and on its insert-only cells no kernel finished within the limits, so they are not scored. On dewiki the score is 0.669 and 0.671, but the slowest repeat of PR over the fastest of the best other kernel is 0.710 and 0.783, so both cells miss the test.

Triangles and 4-Cycles. At $p = 3$, sharing beats the scan by a median of 13% on the new streams and never reaches the test. Pooled over 48 scored cells, the median score at $p = 3$ is 0.965, and the best single cell is 0.700. Cheetah reports that sharing gains least for triangles and more for larger patterns [42]. Our results agree only in part: for triangles sharing helps a little and never by the margin of the test, for 4-cycles it does not help, and for 5-cycles it passes on 28 of 50 cells. The new streams have no $p = 4$ cells, so 4-cycles were timed only on the three evaluation streams.

What Sharing Beats. The kernel that sharing beats is almost always the scan. At $p = 5$, R2 is the best other kernel on 43 of 50 pooled cells, and the mixed table M5 or the recount A5 on the other 7. The wedge tables WT and WJ are the best other kernel on no cell. So the gain measured here is the gain of the shared row over the same kernel without sharing.

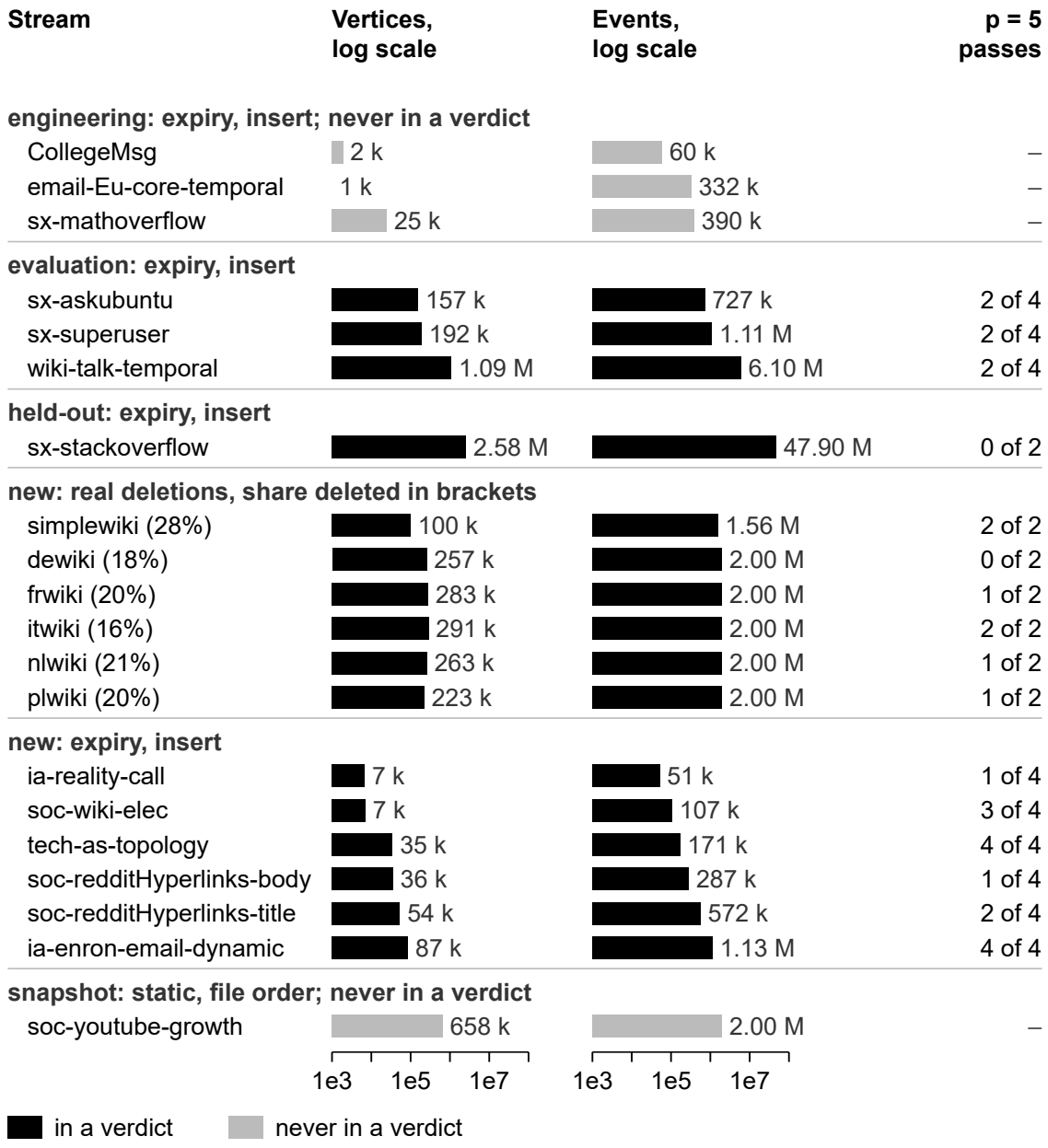


Figure 3: The test set: every stream, grouped by its role and label, with its number of events and its passes at $p = 5$. Streams in black are in a verdict; streams in grey are the engineering streams and the one snapshot stream, which are never scored. The event count is that of the insert-only or real-deletion stream, and an expiry stream has about twice as many. The Wikipedia streams are 2,000,000-event prefixes, with the share of deletions in brackets. sx-stackoverflow is the held-out stream. Pooled, 28 of 50 scored cells pass.

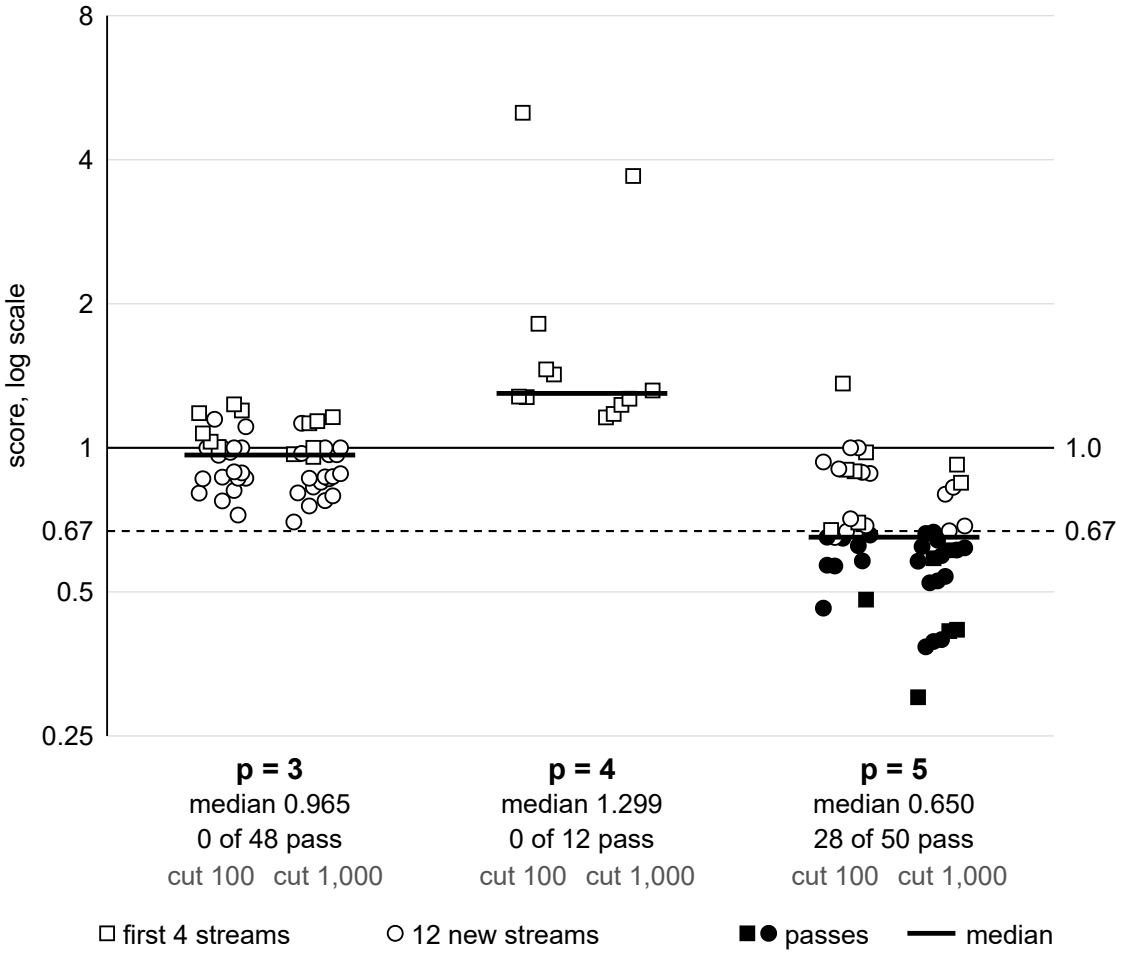


Figure 4: Score against pattern size for every scored cell: the median time of the sharing kernel over the median time of the best other kernel. Squares are cells of the first four streams, which alone have $p = 4$; circles are cells of the twelve new streams. Filled marks pass the test, the slowest repeat of the sharing kernel being at most 0.67 times the fastest repeat of the best other kernel. Bars are medians. Below 1.0 the sharing kernel is faster by median. The snapshot stream is left out.

Set	Label	Scored	Pass	Median score
Evaluation and held-out	insert-only and expiry	14	6	0.686
Twelve new streams	real deletions	12	7	0.646
	synthetic expiry	12	6	0.667
	insert-only	12	9	0.637
Pooled	all	50	28	0.650

Table 3: Cells at $p = 5$. A cell passes when the slowest repeat of PR takes at most 0.67 times the fastest repeat of the best other kernel. The score is the ratio of medians. All streams have real timestamps.

7 Why Operation Counts Mislead: a Cost Model

Two Winners. On 60 of the 76 cells of the twelve new streams and the snapshot stream, the kernel with the fewest operations is not the fastest kernel. On 35 of these cells, at $p = 3$, the scan does the fewest operations and another kernel is faster. On 25, at $p = 5$, a wedge table, the mixed table or the recount does the fewest operations and PR is fastest. The flat wedge table WJ runs at a median of 9.55 ns per counted operation on these streams, against 0.35 ns for PR.

Three Models. We fit three linear models of a kernel's median time on a cell to its operation counts on that cell, by nonnegative least squares. Model U charges one price for every operation. Model K charges one price for each of ten kinds of operation: a list element scanned, a lookup of an edge in the graph's hash map of edges, a row read or write, a degree read, a wedge-table read, a wedge-table write, a graph edit, a step of reducing the batch, a step of grouping it, and a step of a static recount. Model KC adds an extra price for edge lookups and table reads and writes when the kernel's table is larger than the 30 MB last-level cache. The models are fitted on one set of streams and judged on another set that no coefficient has seen, by their median relative error and by how often they name the fastest kernel on a cell. A kernel on a cell is left out when it runs under 0.05 s, since its time is then dominated by process start and the clock's resolution. On the twelve new streams this leaves 64 cells with at least two kernels, on which the fastest kernel can be named.

Fitted on the Engineering Streams. Fitted on the three engineering streams and read on the twelve new streams, model KC names the fastest kernel on 63 of 64 cells, at a median error of 23.0%. Model U names 21 of 64, at a median error of 71.3%. Fitted instead on the first three evaluation streams, model KC names all 64 at 22.9%. Figure 5 shows predicted against measured time for both fits.

The Prices. In both fits, a step along a row or an adjacency list costs 0.05 to 0.6 ns, a wedge-table read about 1 to 2 ns, a wedge-table write 1.5 to 4.9 ns, and a lookup of an edge in the graph's hash map 10 to 18 ns. The operation counts of different kernels are correlated, so a single coefficient is not a measured price of one instruction. In both fits, though, a table read or write costs several times a step along a row or list. This is why unit cost fails: the table kernels replace many steps along lists with fewer table operations, and each of those costs more.

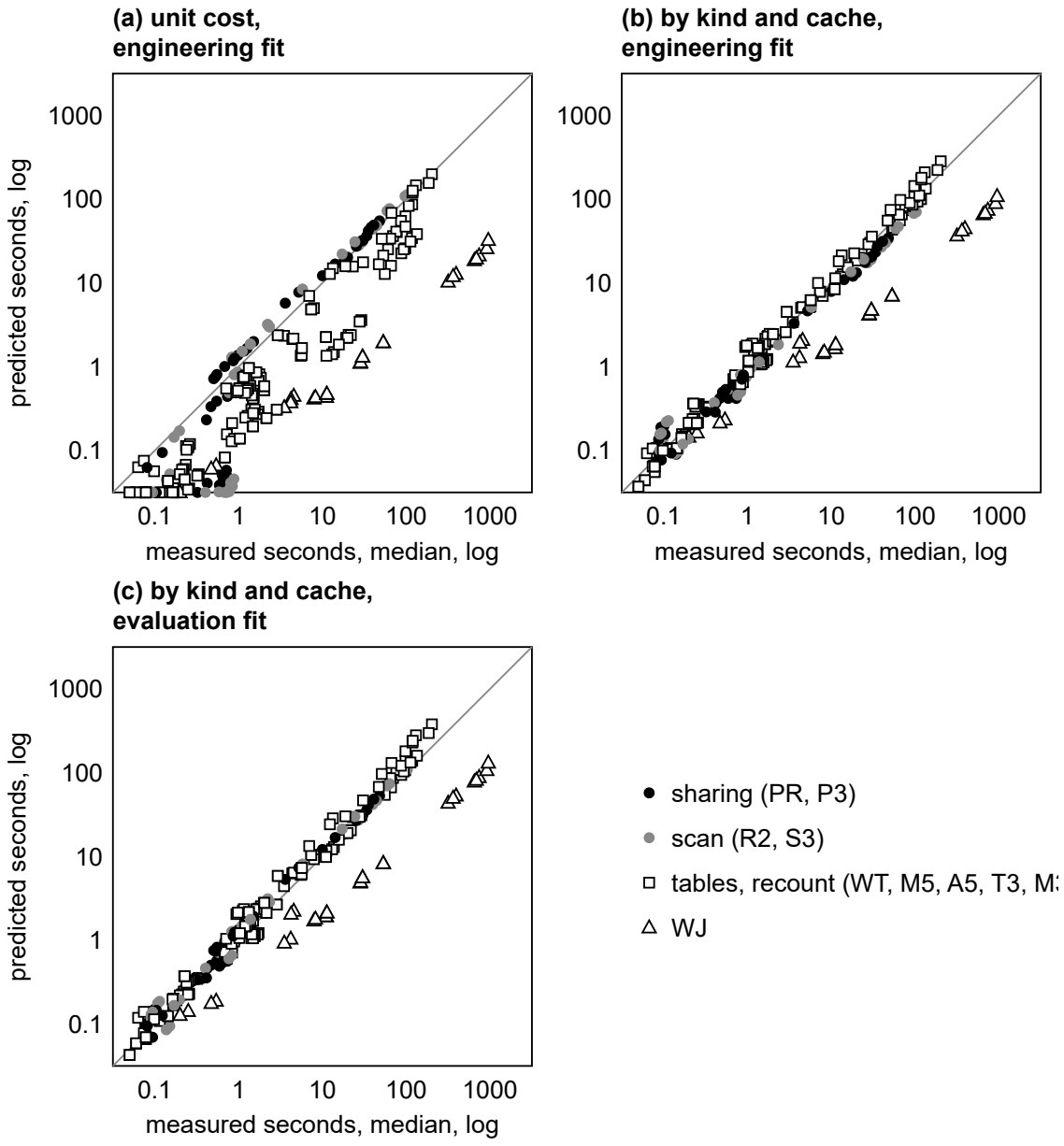


Figure 5: Predicted against measured time on the twelve new streams, whose cells no coefficient saw. Panels (a) and (b) are fitted on the three engineering streams, and panel (c) on the three evaluation streams. The fastest kernel is named on 21, 63 and 64 of 64 cells, at median errors of 71.3%, 23.0% and 22.9%. The extra price for tables larger than the cache adds nothing in either fit, and both fits predict WJ at a median of 0.15 and 0.18 of its time. In (a), 38 predictions under 0.03 s are drawn at the lower edge.

No Cache-Size Effect. The extra prices for tables larger than the cache add nothing in either fit. On the new streams, model KC names the same 63 winners as model K, and lowers the median error only from 23.5% to 23.0%. The fit on the three evaluation streams gives the same picture. So the data do not show that a table outgrowing the last-level cache is what makes the table kernels slow.

What the Model Misses. The model does not explain the flat table. It predicts WJ at a median of 0.15 of its measured time with the engineering fit, and 0.18 with the other. WJ and the tuned table WT count the same kinds of operation and hold the same entries, and WT runs at a median of 1.42 ns per counted operation against 9.55 for WJ. What separates them, the layout of the table in memory, is not in any counter. The engineering fit also fails on the three evaluation streams at $p = 3$ and $p = 4$: there it names the wrong winner on 14 of 24 cells, each time preferring the sharing kernel where the scan, or once the mixed table, is fastest, by margins of 0.3% to 28%. At $p = 5$ it names all 12 correctly.

8 Sensitivity

The Mixed Table at Its Best Threshold. The mixed table was run on the six first passes at every threshold from 16 to 1,024, and the best threshold on each cell was chosen after the fact. All six passes hold against it. The best threshold is 16 on the expiry cells of sx-askubuntu and sx-superuser and 64 on the other cells, and the mixed table beats the scan on one stream only, by 9%. Figure 6 shows the mixed table's time over the threshold against PR and R2.

The Scan Margin. The verdict rests on a margin of about 10% over the scan. If the best other kernel's fastest repeat were 5% faster on every cell, 22 of 50 pooled cells would pass, and at 10% faster 16 of 50, below the third the verdict needs. A faster scan on another machine could therefore change the verdict, and this is the main reason the result is stated for one machine.

Background Load. After each timing run, every timed repeat was checked against the recorded start times of the scheduled tasks on the machine. In the two timing runs of the new streams, overlapping repeats ran a median of 0.00% and 0.13% over their kernel's median on the cell, and clean repeats 0.00%. On three passing cells, every repeat of the best other kernel overlapped the run of a scheduled task. Load can only slow that kernel, so for these passes to be false, the load would have had to slow it by 15% to 57% in every repeat. If the two of these cells that need the smallest slowdown are removed, together with one cell that runs in a few milliseconds, 25 of the remaining 47 cells pass.

Spread of the Repeats. On the new streams, the median ratio of the slowest to the fastest repeat of one kernel on one cell is 1.012 at $p = 3$ and 1.005 at $p = 5$, and the largest is 1.155, for R2 on a Wikipedia stream with real deletions. The pass test compares the slowest repeat of PR with the fastest of the best other kernel, so this spread counts against sharing.

An Efficiency Core. The 24 cells nearest the pass line were timed again on a performance core and on an efficiency core of the same processor. On the performance core, each kernel's median is within 1% of the earlier run, and the pooled count stays at 28 of 50. With the efficiency-core verdicts in place of the recorded ones on these 24 cells, 24 of the 50 pooled cells pass, and every

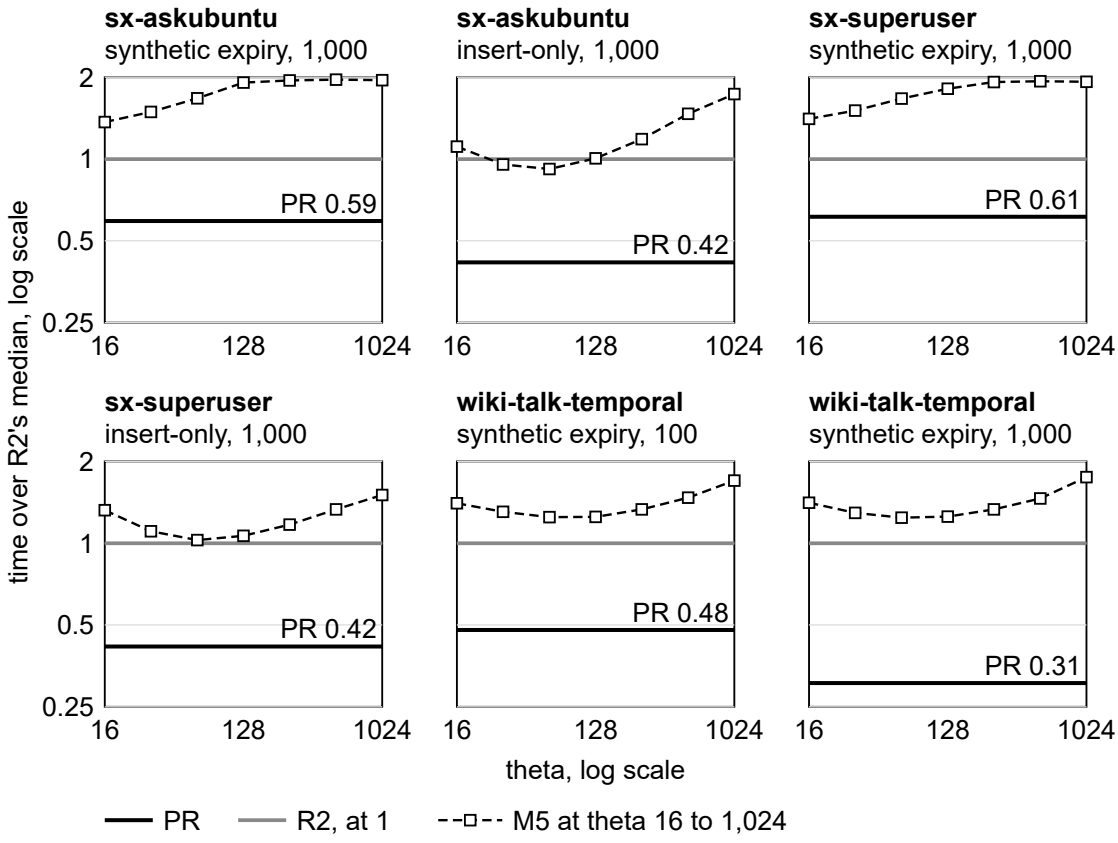


Figure 6: The mixed table M5 at every threshold θ from 16 to 1,024, against PR and R2, on the six cells that pass on the evaluation streams. Each value is a median of the timing repeats over R2's median. At every threshold, PR's slowest repeat stays at most 0.67 times M5's fastest, so all six passes hold against M5 at its best threshold.

cell lost is a Wikipedia stream with real deletions. The efficiency core runs PR and R2 about 1.6 times slower, but not by the same factor on every stream.

9 Limits and Conclusion

One Machine. All timing comes from one machine. The efficiency-core run changes the core and its first two cache levels, and keeps the last-level cache and the memory, so it does not replace a second machine. Given the scan margin of Section 8, a machine with a relatively faster scan could change the verdict at five vertices.

Sixteen Streams. Sixteen real-timestamp streams are few beside the hundreds of instances in static benchmark sets. Streams with real timestamps are scarce, and those with recorded deletions more so. Our streams are the real instances of the set of Hanauer et al. [16], chosen by a rule fixed before their sizes were read. All 18 are used: 16 are scored, one is an engineering stream, and one is a single snapshot and is reported only. The study also stops at batches of 1,000 events and at five vertices, and it is sequential. Every baseline is our own implementation, written so that no kernel loses to a careless form. The published counters closest to ours, those of Hanauer et al., take single updates and stop at four vertices, so they do not run on the cells where sharing passes. We did not add synthetic streams, because a synthetic stream adds instances without adding evidence about real update patterns.

What Is New. The mechanism is not new. DPTL+ and BFCB-IG reuse a row per vertex across a batch [40, 38], and Cheetah groups batches into stars and reports that triangles gain least [42]. What this paper adds is the measured gain of that mechanism against the strongest sequential exact baselines, with every cost inside, on real streams with real deletions and a held-out stream, and a cost model that explains why operation counts choose the wrong kernel.

Conclusion. Sharing work inside a batch pays for exact counting of 5-cycles against the strongest sequential baselines we built, on 28 of 50 cells over 14 of 16 real streams, more often at batches of 1,000 events than of 100. It does not pay for triangles or 4-cycles by the same test. It fails on the held-out stream and on the German Wikipedia stream, and with a scan 10% faster the verdict would not hold. An engineer choosing among these kernels should price each kind of operation rather than count operations. A model priced by kind, fitted on three small streams, names the fastest kernel on 63 of 64 cells of the twelve new streams. Its limits are that it does not explain the flat wedge table, finds no effect of table size, and, on the three evaluation streams, names the wrong kernel on 14 of 24 cells at three and four vertices.

Data and Code Availability

The source code of every kernel, the scripts that prepare the streams, run the measurements and compute every number in this paper, and the per-cell results are available at `[repository to be released on acceptance]`. The input streams are public: SNAP, KONECT and Network Repository, as listed by Hanauer et al. [16].

Use of Generative AI

This study was carried out with an AI assistant, Claude (Anthropic), working under the direction of the corresponding author, who made every decision recorded in the project log. The assistant was used in four places. First, it wrote the source code of every kernel, the exactness tests, the stream-preparation scripts and the measurement and analysis scripts. Second, it wrote the experimental protocols, each committed to version control before the run it governs, and it ran the measurements under those protocols. Third, it searched and read the literature and drafted the bibliography. Fourth, it drafted the text of this manuscript and drew its figures. The output was verified as follows. Every kernel's count was checked against a brute-force count on random graphs, with deliberately broken versions of each kernel required to fail, and every kernel's count on every batch of every stream was checked against every other kernel's. Every number in the manuscript was traced to a committed result file. Every bibliography entry was checked against the publisher's page or the DOI record. The authors read the full manuscript, and the corresponding author changed every sentence that the authors would not state as their own. The authors take full responsibility for the content.

Acknowledgements

The authors thank Universiti Teknikal Malaysia Melaka (UTeM) for its support.

References

[1] N. Alon, R. Yuster, and U. Zwick. Finding and counting given length cycles. *Algorithmica*, 17(3):209–223, 1997. doi:10.1007/BF02523189.

[2] K. Ammar, F. McSherry, S. Salihoglu, and M. Joglekar. Distributed evaluation of subgraph queries using worst-case optimal low-memory dataflows. *Proceedings of the VLDB Endowment*, 11(6):691–704, 2018. doi:10.14778/3184470.3184473.

[3] L. Bindschaedler, J. Malicevic, B. Lepers, A. Goel, and W. Zwaenepoel. Tesseract: Distributed, general graph pattern mining on evolving graphs. In *Proceedings of the Sixteenth European Conference on Computer Systems (EuroSys '21)*, pages 458–473. ACM, 2021. doi:10.1145/3447786.3456253.

[4] J. Borowitz, E. Großmann, C. Schulz, and D. Schweisgut. Scalable algorithms for 2-packing sets on arbitrary graphs. *Journal of Graph Algorithms and Applications*, 29(1):159–186, 2025. doi:10.7155/jgaa.v29i1.3064.

[5] R. Cannoodt, J. Ruyssinck, J. Ramon, K. De Preter, and Y. Saeys. IncGraph: Incremental graphlet counting for topology optimisation. *PLOS ONE*, 13(4):e0195997, 2018. doi:10.1371/journal.pone.0195997.

[6] D. Chakrabarti, Y. Zhan, and C. Faloutsos. R-MAT: A recursive model for graph mining. In *Proceedings of the 2004 SIAM International Conference on Data Mining (SDM)*, pages 442–446. SIAM, 2004. doi:10.1137/1.9781611972740.43.

[7] F. Chanchary and A. Maheshwari. Time windowed data structures for graphs. *Journal of Graph Algorithms and Applications*, 23(2):191–226, 2019. doi:10.7155/jgaa.00489.

[8] Q. Chen, O. Lachish, S. Helmer, and M. H. Böhlen. Dynamic spanning trees for connectivity queries on fully-dynamic undirected graphs. *Proceedings of the VLDB Endowment*, 15(11):3263–3276, 2022. doi:10.14778/3551793.3551868.

[9] N. Chiba and T. Nishizeki. Arboricity and subgraph listing algorithms. *SIAM Journal on Computing*, 14(1):210–223, 1985. doi:10.1137/0214017.

[10] L. Dhulipala, Q. C. Liu, J. Shun, and S. Yu. Parallel batch-dynamic k-clique counting. In *Symposium on Algorithmic Principles of Computer Systems (APOCS)*, pages 129–143. SIAM, 2021. doi:10.1137/1.9781611976489.10.

[11] D. Ediger, K. Jiang, J. Riedy, and D. A. Bader. Massive streaming data analytics: A case study with clustering coefficients. In *2010 IEEE International Symposium on Parallel and Distributed Processing, Workshops and PhD Forum (IPDPSW)*, pages 1–8. IEEE, 2010. doi:10.1109/IPDPSW.2010.5470687.

[12] D. Eppstein, M. T. Goodrich, D. Strash, and L. Trott. Extended dynamic subgraph statistics using h-index parameterized data structures. *Theoretical Computer Science*, 447:44–52, 2012. doi:10.1016/j.tcs.2011.11.034.

[13] D. Eppstein and E. S. Spiro. The h-index of a graph and its application to dynamic subgraph statistics. *Journal of Graph Algorithms and Applications*, 16(2):543–567, 2012. doi:10.7155/jgaa.00273.

[14] S. D. Fink and I. Rutter. Constrained planarity in practice – engineering the synchronized planarity algorithm. *Journal of Graph Algorithms and Applications*, 29(1):91–123, 2025. doi:10.7155/jgaa.v29i1.2923.

[15] E. Großmann, F. Joos, H. Reinstädtler, and C. Schulz. Engineering hypergraph b-matching algorithms. *Journal of Graph Algorithms and Applications*, 30(1):1–24, 2026. doi:10.7155/jgaa.v30i1.3166.

[16] K. Hanauer, S. Heck, M. Henzinger, and L. Sidl. Fully dynamic triangle and 4-vertex subgraph counting: From theory to practice and back again. arXiv:2610.29776, 2026. Short version to appear at ALENEX 2027. URL: https://arxiv.org/abs/2610.29776, arXiv:2610.29776, doi:10.48550/arXiv.2610.29776.

[17] K. Hanauer, M. Henzinger, and Q. C. Hua. Fully dynamic four-vertex subgraph counting. In J. Aspnes and O. Michail, editors, *1st Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2022)*, volume 221 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 18:1–18:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi:10.4230/LIPIcs.SAND.2022.18.

[18] K. Hanauer, M. Henzinger, and C. Schulz. Recent advances in fully dynamic graph algorithms – a quick reference guide. *ACM Journal of Experimental Algorithmics*, 27:1–45, 2022. doi:10.1145/3555806.

[19] A. Kara, H. Q. Ngo, M. Nikolic, D. Olteanu, and H. Zhang. Maintaining triangle queries under updates. *ACM Transactions on Database Systems*, 45(3):1–46, 2020. doi:10.1145/3396375.

[20] A. Khan, N. Tan, J. Marquez, M. Taufer, and S. Bhowmick. ParaDyMS: Parallel dynamic motif counting at scale. In *2025 IEEE 25th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 1–10. IEEE, 2025. doi:10.1109/CCGRID64434.2025.00046.

[21] S. Kumar, W. L. Hamilton, J. Leskovec, and D. Jurafsky. Community interaction and conflict on the web. In *Proceedings of the 2018 World Wide Web Conference (WWW '18)*, pages 933–943, 2018. doi:10.1145/3178876.3186141.

[22] J. Kunegis. KONECT: The Koblenz network collection. In *Proceedings of the 22nd International Conference on World Wide Web (WWW '13 Companion)*, pages 1343–1350. ACM, 2013. doi:10.1145/2487788.2488173.

[23] J. Leskovec, D. Huttenlocher, and J. Kleinberg. Governance in social media: A case study of the Wikipedia promotion process. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, volume 4, pages 98–105, 2010. doi:10.1609/icwsm.v4i1.14013.

[24] J. Leskovec and A. Krevl. SNAP datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, June 2014.

[25] Q. C. Liu, J. Shi, S. Yu, L. Dhulipala, and J. Shun. Parallel batch-dynamic algorithms for k-core decomposition and related graph problems. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 191–204. ACM, 2022. doi:10.1145/3490148.3538569.

[26] S. Lu and Y. Tao. Towards optimal dynamic indexes for approximate (and exact) triangle counting. In K. Yi and Z. Wei, editors, *24th International Conference on Database Theory (ICDT 2021)*, volume 186 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 6:1–6:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.ICDT.2021.6.

[27] D. Makkar, D. A. Bader, and O. Green. Exact and parallel triangle counting in dynamic graphs. In *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*, pages 2–12. IEEE, 2017. doi:10.1109/HiPC.2017.00011.

[28] A. Mhedhbi, C. Kankanamge, and S. Salihoglu. Optimizing one-time and continuous subgraph queries using worst-case optimal joins. *ACM Transactions on Database Systems*, 46(2):1–45, 2021. doi:10.1145/3446980.

[29] P. Panzarasa, T. Opsahl, and K. M. Carley. Patterns and dynamics of users' behavior and interaction: Network analysis of an online community. *Journal of the American Society for Information Science and Technology*, 60(5):911–932, 2009. doi:10.1002/asi.21015.

[30] A. Paranjape, A. R. Benson, and J. Leskovec. Motifs in temporal networks. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*, pages 601–610. ACM, 2017. doi:10.1145/3018661.3018731.

[31] A. Pinar, C. Seshadhri, and V. Vishal. ESCAPE: Efficiently counting all 5-vertex subgraphs. In *Proceedings of the 26th International Conference on World Wide Web (WWW)*, pages 1431–1440. ACM, 2017. doi:10.1145/3038912.3052597.

[32] J. Preusse, J. Kunegis, M. Thimm, S. Staab, and T. Gottron. Structural dynamics of knowledge networks. In *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, volume 7, pages 506–515, 2013. doi:10.1609/icwsm.v7i1.14402.

[33] L. Qiu, L. Chen, H. Jie, X. Ke, Y. Gao, Y. Liu, and Z. Zhang. GPU-accelerated batch-dynamic subgraph matching. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3204–3216. IEEE, 2024. doi:10.1109/ICDE60146.2024.00248.

[34] R. A. Rossi and N. K. Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, volume 29, pages 4292–4293. AAAI Press, 2015. doi:10.1609/aaai.v29i1.9277.

[35] J. Shi, L. R. Huang, and J. Shun. Parallel five-cycle counting algorithms. *ACM Journal of Experimental Algorithmics*, 27:1–23, 2022. doi:10.1145/3556541.

[36] S. Sun, X. Sun, B. He, and Q. Luo. RapidFlow: An efficient approach to continuous subgraph matching. *Proceedings of the VLDB Endowment*, 15(11):2415–2427, 2022. doi:10.14778/3551793.3551803.

[37] X. Sun, S. Sun, Q. Luo, and B. He. An in-depth study of continuous subgraph matching. *Proceedings of the VLDB Endowment*, 15(7):1403–1416, 2022. doi:10.14778/3523210.3523218.

[38] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang. Accelerated butterfly counting with vertex priority on bipartite graphs. *The VLDB Journal*, 32(2):257–281, 2023. Published online 2022. doi:10.1007/s00778-022-00746-0.

[39] Z. Wang, J. Tang, Z. Li, J. Tu, W. Xue, and J. Huang. EDTC: Exact triangle counting for dynamic graphs on GPU. *IEEE Transactions on Parallel and Distributed Systems*, 37(1):247–259, 2026. doi:10.1109/TPDS.2025.3627974.

[40] M. Yu, L. Qin, Y. Zhang, W. Zhang, and X. Lin. DPTL+: Efficient parallel triangle listing on batch-dynamic graphs. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 1332–1343. IEEE, 2021. doi:10.1109/ICDE51399.2021.00119.

[41] Y. Zhang, Y. Huang, C. Liu, H. Liu, J. Chen, J. Yuan, J. Yue, X. Liao, H. Jin, and J. Xue. Gopher: Efficient dynamic graph pattern mining via DAG-driven execution. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*, pages 1722–1737. ACM, 2026. doi:10.1145/3767295.3769365.

[42] Y. Zhang, X. Yi, Y. Huang, J. Yuan, C. Gui, D. Chen, L. Zheng, J. Yue, X. Liao, H. Jin, and J. Xue. Cheetah: Accelerating dynamic graph mining with grouping updates. *ACM Transactions on Architecture and Code Optimization*, 22(2):1–26, 2025. doi:10.1145/3736173.

A Every Scored Cell at $p = 5$

Table 4 lists the 50 scored cells at $p = 5$. The first 14 rows are the three evaluation streams and the held-out stream sx-stackoverflow; the other 36 are the twelve new streams. Seconds are medians of the timing repeats. The score is the ratio of the medians, and the test is the slowest repeat of PR over the fastest repeat of the best other kernel, which passes at 0.67 or less. A best kernel marked * was stopped at its time limit and enters at that limit.

Stream	Label	Batch	PR s	Best	Best s	Score	Test	Pass
sx-askubuntu	synthetic expiry	1,000	0.268	R2	0.456	0.588	0.592	yes
sx-askubuntu	synthetic expiry	100	0.429	R2	0.480	0.894	0.900	no
sx-askubuntu	insert-only	1,000	4.866	M5	11.760	0.414	0.417	yes
sx-askubuntu	insert-only	100	8.183	M5	11.717	0.698	0.708	no
sx-superuser	synthetic expiry	1,000	0.469	R2	0.768	0.611	0.616	yes
sx-superuser	synthetic expiry	100	0.719	R2	0.799	0.900	0.929	no
sx-superuser	insert-only	1,000	13.254	R2	31.793	0.417	0.421	yes
sx-superuser	insert-only	100	21.417	R2	31.815	0.673	0.678	no
wiki-talk-temporal	synthetic expiry	1,000	10.921	R2	36.261	0.301	0.305	yes
wiki-talk-temporal	synthetic expiry	100	17.519	R2	36.386	0.481	0.487	yes
wiki-talk-temporal	insert-only	1,000	276.627	A5*	300.000	0.922	0.955	no
wiki-talk-temporal	insert-only	100	408.891	A5*	300.000	1.363	1.380	no
sx-stackoverflow	synthetic expiry	1,000	130.920	R2	154.827	0.846	0.863	no
sx-stackoverflow	synthetic expiry	100	153.099	R2	156.560	0.978	0.983	no
dewiki	real deletions	100	20.00	R2	29.89	0.669	0.710	no
dewiki	real deletions	1,000	18.21	R2	27.15	0.671	0.783	no
frwiki	real deletions	100	33.93	R2	47.74	0.711	0.725	no
frwiki	real deletions	1,000	29.04	R2	43.84	0.662	0.665	yes
itwiki	real deletions	100	47.97	R2	103.82	0.462	0.482	yes
itwiki	real deletions	1,000	38.12	R2	96.73	0.394	0.426	yes
nlwiki	real deletions	100	30.49	R2	44.36	0.687	0.692	no
nlwiki	real deletions	1,000	25.37	R2	39.57	0.641	0.643	yes
plwiki	real deletions	100	41.10	R2	63.23	0.650	0.670	no
plwiki	real deletions	1,000	35.21	R2	59.16	0.595	0.596	yes
simplewiki	real deletions	100	14.14	R2	24.36	0.581	0.591	yes
simplewiki	real deletions	1,000	9.99	R2	17.24	0.580	0.614	yes
ia-enron-email-dynamic	synthetic expiry	100	0.55	R2	0.97	0.568	0.577	yes
ia-enron-email-dynamic	synthetic expiry	1,000	0.47	R2	0.89	0.527	0.533	yes
ia-enron-email-dynamic	insert-only	100	1.26	R2	2.23	0.567	0.571	yes
ia-enron-email-dynamic	insert-only	1,000	1.16	R2	2.22	0.523	0.531	yes
ia-reality-call	synthetic expiry	100	0.01	R2	0.01	1.000	1.000	no
ia-reality-call	synthetic expiry	1,000	0.00	R2	0.01	0.800	0.800	no
ia-reality-call	insert-only	100	0.00	R2	0.00	1.000	1.000	no
ia-reality-call	insert-only	1,000	0.00	R2	0.00	0.667	0.667	yes
soc-redditHyperlinks-body	synthetic expiry	100	0.14	R2	0.15	0.934	0.940	no
soc-redditHyperlinks-body	synthetic expiry	1,000	0.09	R2	0.14	0.686	0.691	no
soc-redditHyperlinks-body	insert-only	100	0.98	R2	1.10	0.888	0.889	no
soc-redditHyperlinks-body	insert-only	1,000	0.68	R2	1.10	0.618	0.625	yes
soc-redditHyperlinks-title	synthetic expiry	100	0.73	R2	0.83	0.884	0.889	no
soc-redditHyperlinks-title	synthetic expiry	1,000	0.41	R2	0.77	0.539	0.544	yes
soc-redditHyperlinks-title	insert-only	100	5.22	R2	5.77	0.904	0.905	no
soc-redditHyperlinks-title	insert-only	1,000	3.59	R2	5.77	0.622	0.633	yes
soc-wiki-elec	synthetic expiry	100	0.12	R2	0.20	0.624	0.628	yes
soc-wiki-elec	synthetic expiry	1,000	0.08	A5	0.10	0.827	0.845	no
soc-wiki-elec	insert-only	100	0.55	R2	0.84	0.657	0.657	yes
soc-wiki-elec	insert-only	1,000	0.51	R2	0.83	0.611	0.614	yes
tech-as-topology	synthetic expiry	100	1.51	R2	2.33	0.647	0.649	yes
tech-as-topology	synthetic expiry	1,000	0.90	R2	2.33	0.384	0.385	yes
tech-as-topology	insert-only	100	0.86	M5	1.32	0.651	0.657	yes
tech-as-topology	insert-only	1,000	0.53	M5	1.33	0.397	0.400	yes

Table 4: The 50 scored cells at $p = 5$, 28 of which pass. All streams have real timestamps.